

Lab 6 Report

Nima Akbari

501322748

COE 328: Digital Systems

Section 17

Dr. Reza Sedaghat

December 1, 2025

Table of Contents

Cover Page	1
Table of Contents	2
Introduction	3
Components	4
Problem 1	11
Problem 2	13
Problem 3	16
Conclusion	19

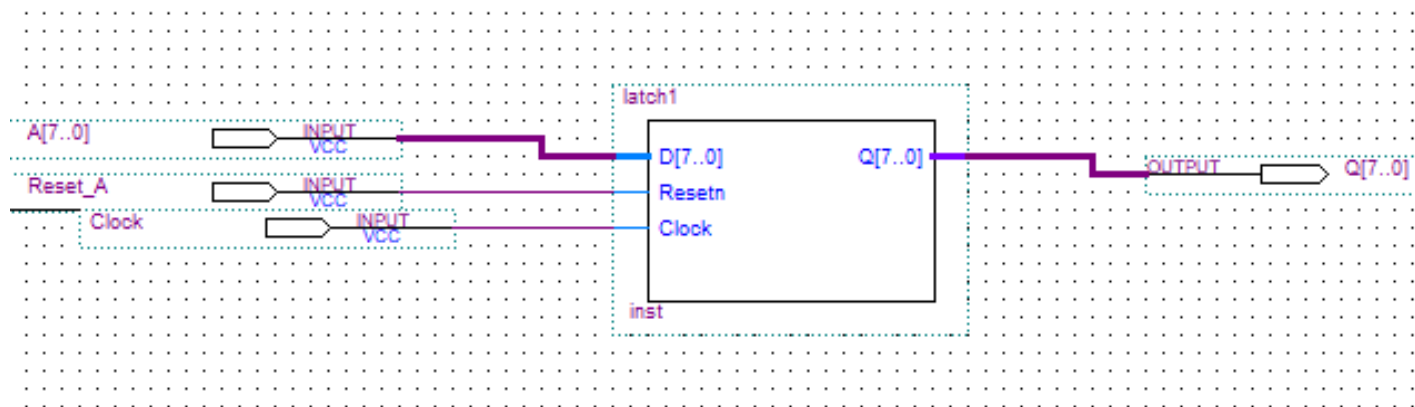
Introduction

In this, we focus on designing a simple processor by building and integrating four major digital components. We use two storage registers(Latch), a control unit(FSM), and an Arithmetic Logic Unit(ALU). The registers store the 8-bit inputs of A and B, which are derived from the last four digits of our student ID. The control unit consists of a Finite State Machine(FSM) and 2 3-to-8 decoders, which create a 4-to-16 decoder. The FSM generates a 4-bit current state, which the decoder expands into a 15-bit microcode used to select the ALU operations. Depending on the microcode the ALU receives, it performs arithmetic and logical functions using the values of A and B. These results are displayed using seven-segment displays in hexadecimal format. Across all three problems, we modify the functionality of the ALU and the FSM to demonstrate how processor operations can be customised.

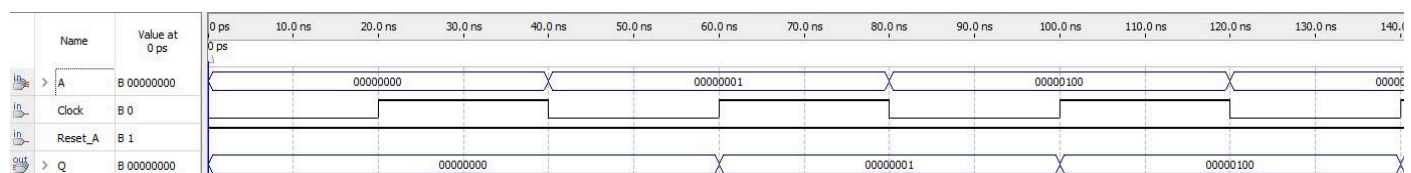
Components

Latch

The latch is an 8-bit storage element which is used to hold the input values of A and B before they enter the ALU. Each latch samples its input on the rising edge of the clock and stores that value until the next clock pulse, which allows the data to be passed into the system more stable. When the reset signal is active, the output of the latch is cleared to 00000000. In our lab, we use two latches, one for A and one for B, and they provide the ALU with consistent inputs for the operations.



Latch1/2 block diagram



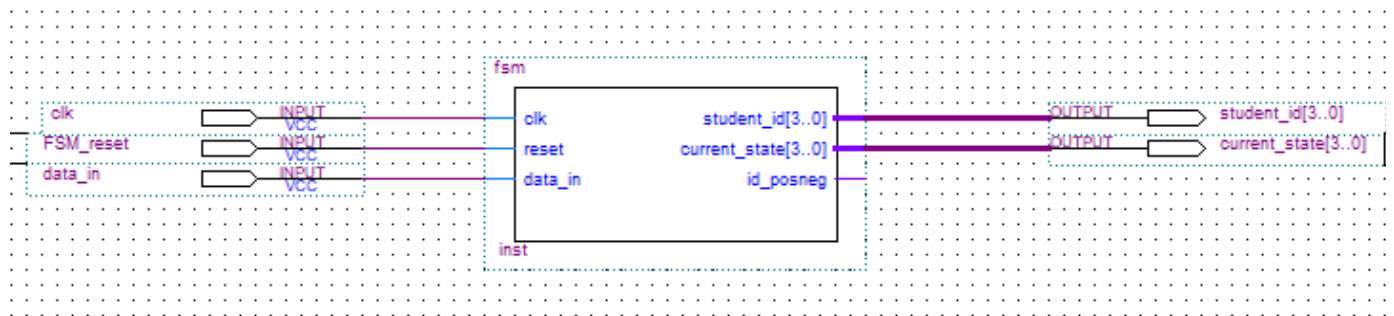
Latch1/2 waveform

Clock	Reset_A	A[7..0]	Q[7..0]
0	x	x	x
1	0	x	00000000
1	1	00000000	00000000
1	1	00000001	00000001
1	1	00000010	00000010
1	1	00000011	00000011
1	1	00000100	00000100
1	1	00000101	00000101
1	1	00000110	00000110
1	1	00000111	00000111
1	1	00001000	00001000
1	1	...	...
1	1	11111110	11111110
1	1	11111111	11111111

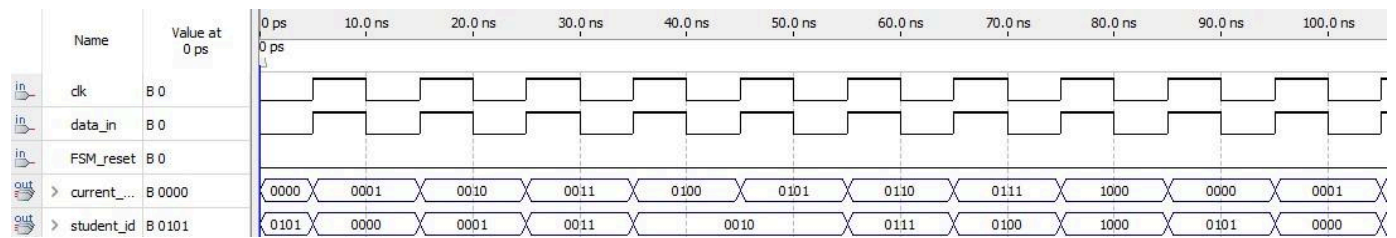
Latch1/2 truth table

FSM

The FSM is a Moore-type state machine which cycles through nine states (S0-S8) and outputs the current_state value to the 4-16 decoder. On each rising edge of the clock, the FSM moves to the next state in the sequence and resets back to state 0 when required. The FSM also outputs the student_id, which is used in problem 3. The FSM's purpose is to control which ALU operation is selected by generating a 4-bit state value for the decoder.



FSM Block Diagram



FSM Waveform

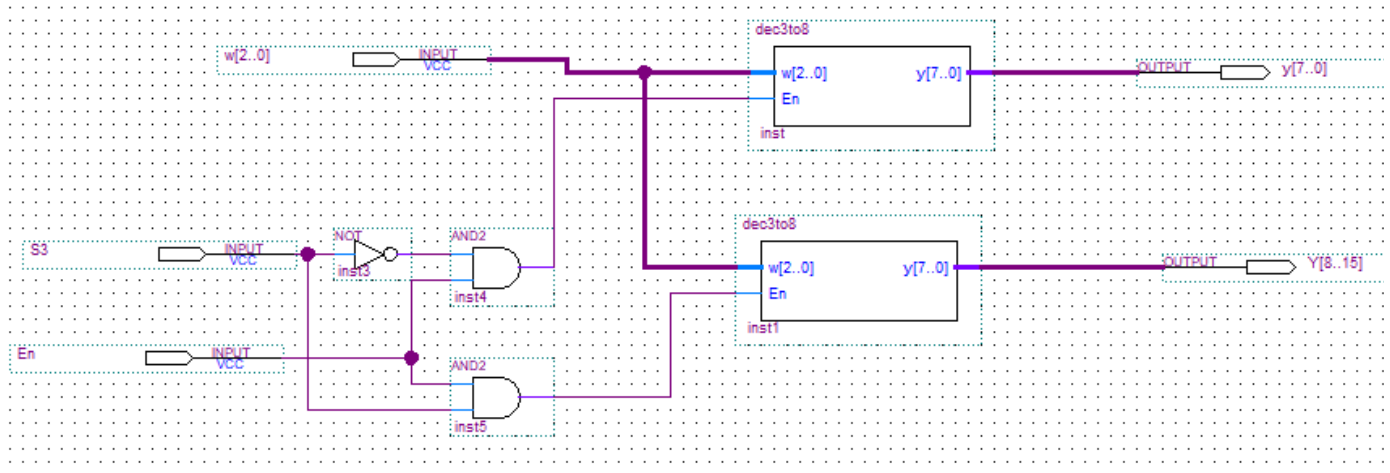
clk	Data_in	FSM_reset	Student_id	Current_state
0	x	x	x	x
1	0	0	No change	No change
1	1	1	0101	0000
1	1	0	0000	0001
1	1	0	0001	0010
1	1	0	0011	0011
1	1	0	0010	0100
1	1	0	0010	0101
1	1	0	0111	0110
1	1	0	0100	0111
1	1	0	1000	1000

1	1	0	0101	0000 (repeats)
---	---	---	------	----------------

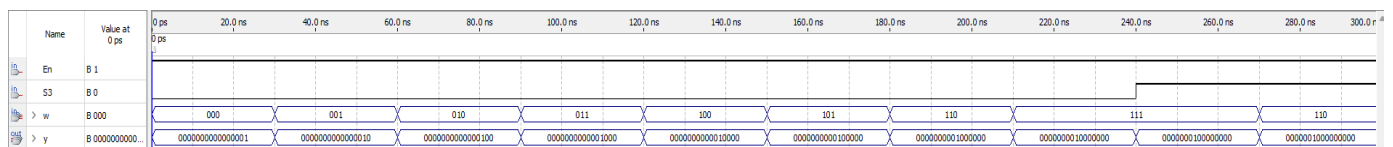
FSM Truth table

4-to-16 Decoder

The 4-to-16 decoder takes a 4-bit input from the FSM and converts it into a 16-bit microcode which is used by the ALU. Instead of using a hard coded 4-to-16 decoder, which slows down processing, we were instructed to build it using two 3-to-8 decoders. The MSB (S3) selects which 3-to-8 decoder is active, and the remaining bits (w2-w0) determine which output goes high. This structure is faster as it only asserts one of the outputs(y0-y15) at a time, and that becomes opcode and is sent to the ALU.



Decoder Block Diagram



Decoder waveform

En	w[2..0]	S3	y[15..0]
0	0	0	0
1	000	0	0000000000000001
1	001	0	0000000000000010
1	010	0	0000000000000100

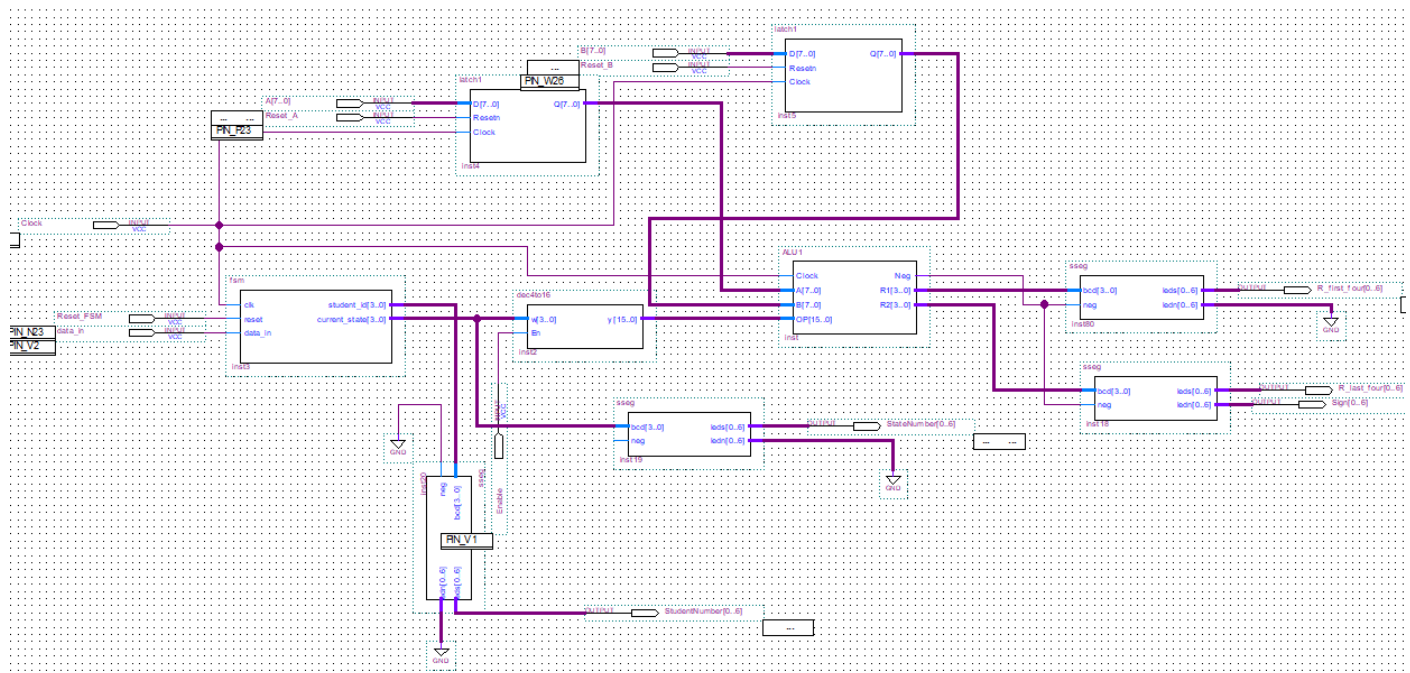
1	011	0	0000000000001000
1	100	0	00000000000010000
1	101	0	00000000000100000
1	110	0	00000000001000000
1	111	0	0000000010000000
1	111	1	0000000100000000
1	110	1	0000001000000000
1	101	1	0000010000000000
1	100	1	0000100000000000
1	011	1	0001000000000000
1	010	1	0010000000000000
1	001	1	0100000000000000
1	000	1	1000000000000000

Decoder Truth Table

Problem 1

Description

Problem 1 uses all four components to implement the initial General Purpose Processor Unit (GPU). It comprises of two 8-bit latches, an FSM, a 4-to-16 decoder created from two 3-to-8 decoders and the ALU core. This design is used as a baseline operation of the ALU before modifications are made to the ALU and the FSM. The FSM counts through states 0-8, and the decoder converts each state into a 16-bit microcode. This microcode selects which ALU operation runs at the clock cycle. The values used in this input are from the last 4 digits of my student number, 2748. A = 27, which converts to 00100111. B = 48, which converts to 01001000.



Problem 1 Block Diagram (ALU1)

Purpose of Inputs & Outputs

Inputs:

- A[7..0] - First 8-bit Operand for the ALU operations, stored in latch 1
- B[7..0] - second 8-bit Operand for the ALU operations, stored in latch 2
- Clock - Controls latches and FSM, determines when there are new values
- Reset_A/Reset_B - Clears latches for A and B to 0, resets
- Reset_FSM - Resets FSM back to state 0
- Enable - Enables the registers to input to A and B
- data_in - Used to change the state of the FSM when the Clock has a rising edge.

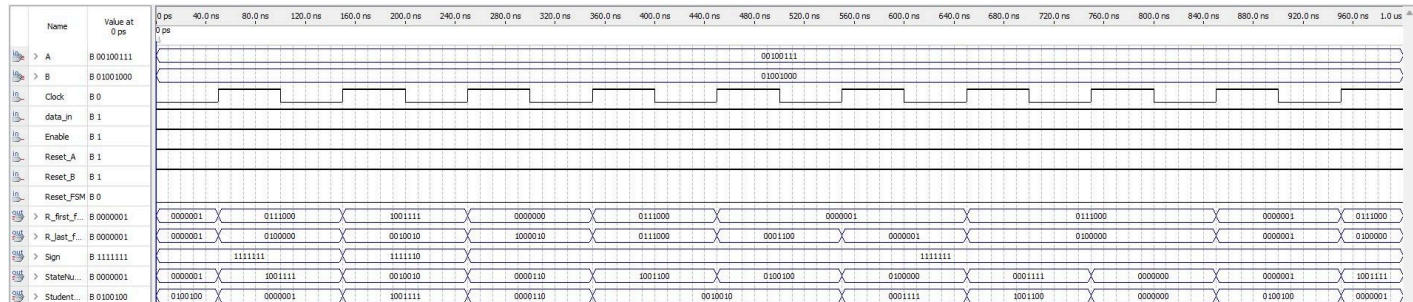
Outputs:

- R_first_four - Lower 4 bits of ALU result in HEX display output
- R_last_four - Upper 4 bits of ALU result in HEX display output
- Sign - MSB of ALU result, displays negative
- current_state[3:0] – FSM state sent into decoder.
- student_id[3:0] – FSM output used for 7-segment display in Problem 1.

Clock	data_in	Reset_FSM	Enable	w[3..0]	y[15..0]	ALU Operations
0	x	x	x	-	-	
1	x	1	x	x	x	
1	1	0	0	0	0	
1	1	0	1	0000	0000000000000001	sum(A,B)
1	1	0	1	0001	0000000000000010	diff(A,B)
1	1	0	1	0010	0000000000000100	Not($A \cdot B$)
1	1	0	1	0011	0000000000001000	Not($A + B$)
1	1	0	1	0100	0000000000010000	$A \cdot B$

1	1	0	1	0101	0000000000100000	$A \oplus B$
1	1	0	1	0110	0000000001000000	$A + B$
1	1	0	1	0111	0000000010000000	$\text{Not}(A \oplus B)$

Table of Microcode's generated by Problem 1 (ALU1)

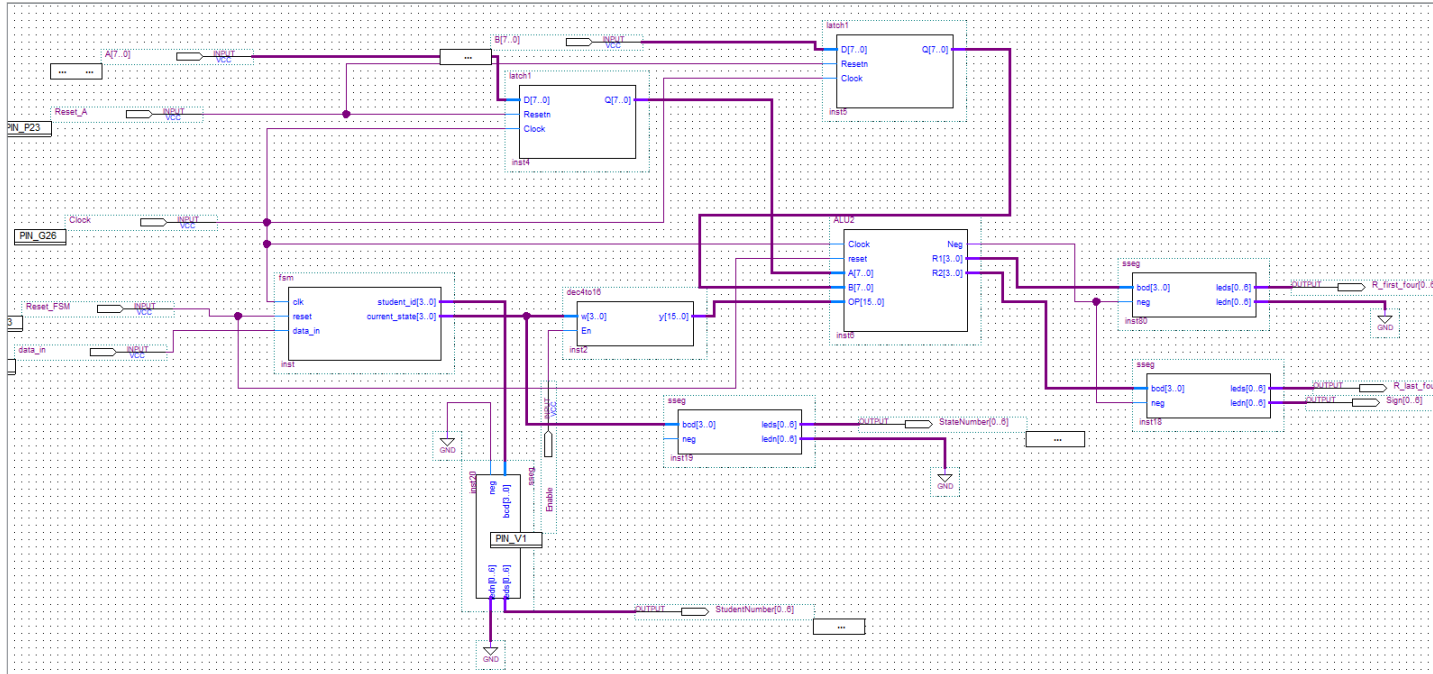


Problem 1 Waveform (ALU1)

Problem 2

Description

Problem 2 extends off the design of problem 1 by integrating the same FSM, same 2 latches, same decoder, however, it expands on the functions of the ALU. We create a new ALU, based on the assigned problem set by our TA. The assigned problem set by our TA was problem set A, which does a new set of functions which we must integrate into our ALU2. The values used in this input are from the last 4 digits of my student number, 2748. A = 27, which converts to 00100111. B = 48, which converts to 01001000.



Problem 2 Block Diagram (ALU2)

Purpose of Inputs & Outputs

Inputs:

- A[7..0] - First 8-bit Operand for the ALU2 operations, stored in latch 1
- B[7..0] - second 8-bit Operand for the ALU2 operations, stored in latch 2
- Clock - Controls latches and FSM, determines when there are new values
- Reset_A/Reset_B - Clears latches for A and B to 0, resets
- Reset_FSM - Resets FSM back to state 0
- Enable - Enables the registers to input to A and B
- data_in - Used to change the state of the FSM when the Clock has a rising edge.

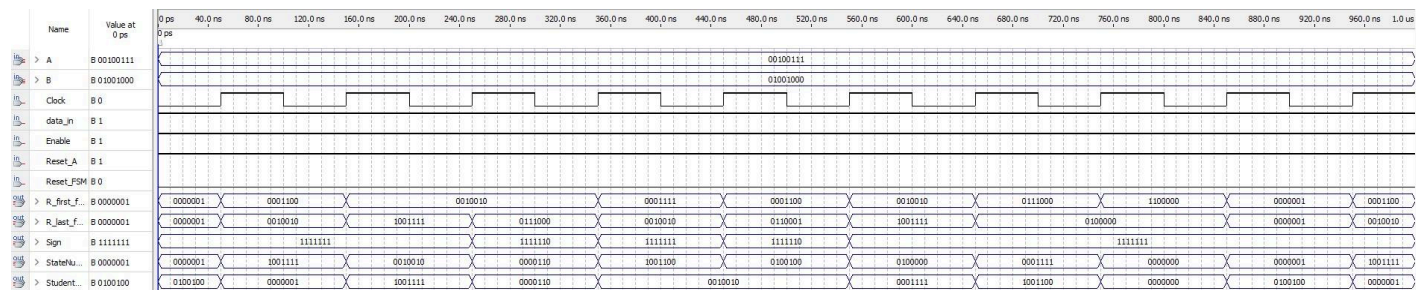
Outputs:

- R_first_four - Lower 4 bits of ALU2 result in HEX display output
- R_last_four - Upper 4 bits of ALU2 result in HEX display output
- Sign - MSB of ALU2 result, displays negative
- current_state[3:0] – FSM state sent into decoder.

- student_id[3:0] – FSM output used for 7-segment display in Problem 1.

Clock	data_in	Reset_FSM	Enable	w[3..0]	y[15..0]	ALU2 Operations (SET A)
0	x	x	x	-	-	-
1	x	1	x	x	x	-
1	1	0	0	0	0	-
1	1	0	1	0000	0000000000000001	Increment A by 2
1	1	0	1	0001	0000000000000010	Shift B to right by two bits, input bit = 0
1	1	0	1	0010	0000000000000100	Shift A to right by four bits, input bit = 1
1	1	0	1	0011	0000000000001000	Find the smaller value of A and B and produce the results (Min(A,B))
1	1	0	1	0100	0000000000010000	Rotate A to right by two bits (ROR)
1	1	0	1	0101	0000000000100000	Invert the bit-significance order of B
1	1	0	1	0110	0000000001000000	Produce the result of XORing A and B
1	1	0	1	0111	0000000010000000	Produce the summation of A and B, then decrease it by 4
1	1	0	1	1000	0000000100000000	Produce all high bits on the output

Table of Microcode's generated by Problem 2 (ALU2)

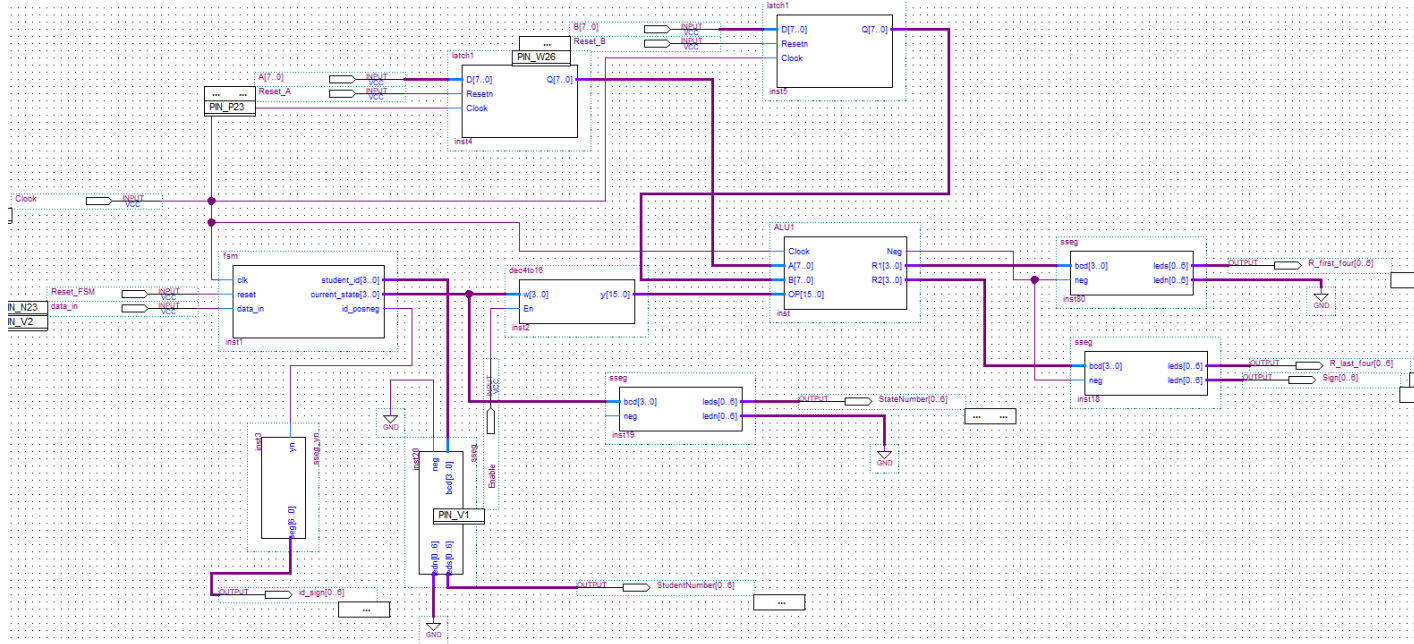


Problem 2 Waveform (ALU2)

Problem 3

Description

Problem 3 extends off the design of problem 1 by integrating the same ALU, same 2 latches, same decoder, however, it expands on the functions of the FSM. We create a new FSM, based on the assigned problem set by our TA. The assigned problem set by our TA was problem set A, which does a new set of functions which we must integrate into our FSM2. The values used in this input are from the last 4 digits of my student number, 2748. A = 27, which converts to 00100111. B = 48, which converts to 01001000.



Problem 3 Block Diagram (ALU1, FSM2)

Purpose of Inputs & Outputs

Inputs:

- A[7..0] - First 8-bit Operand for the ALU2 operations, stored in latch 1
- B[7..0] - second 8-bit Operand for the ALU2 operations, stored in latch 2
- Clock - Controls latches and FSM, determines when there are new values
- Reset_A/Reset_B - Clears latches for A and B to 0, resets
- Reset_FSM - Resets FSM back to state 0
- Enable - Enables the registers to input to A and B
- data_in - Used to change the state of the FSM when the Clock has a rising edge.

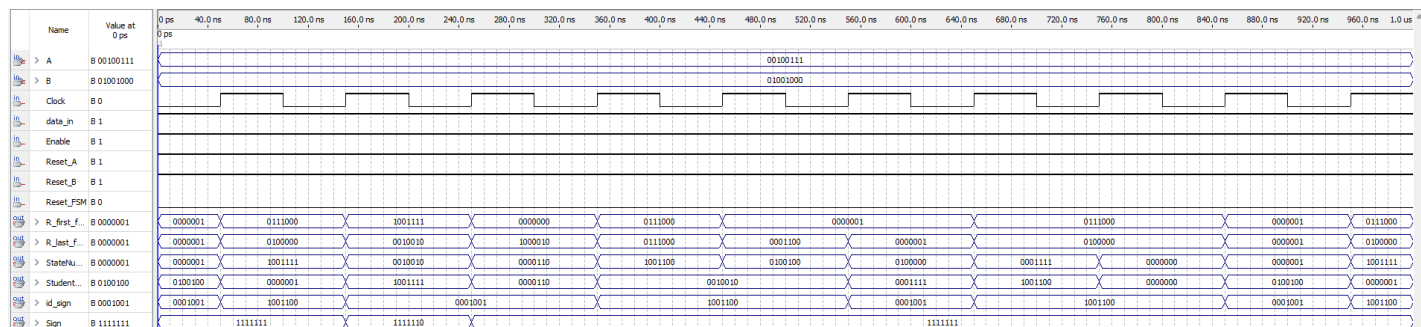
Outputs:

- R_first_four - Lower 4 bits of ALU2 result in HEX display output
- R_last_four - Upper 4 bits of ALU2 result in HEX display output
- Sign - MSB of ALU2 result, displays negative
- current_state[3..0] – FSM state sent into decoder.

- student_id[3..0] – FSM output used for 7-segment display in Problem 1.
- id_sign[0..6] – FSM output used for 7-segment display in Problem 3

Clock	data_in	Reset_FSM	Enable	w[3..0]	y[15..0]	ALU Operations
0	x	x	x	-	-	
1	x	1	x	x	x	
1	1	0	0	0	0	
1	1	0	1	0000	0000000000000001	sum(A,B)
1	1	0	1	0001	0000000000000010	diff(A,B)
1	1	0	1	0010	0000000000000100	Not($A \cdot B$)
1	1	0	1	0011	0000000000001000	Not($A + B$)
1	1	0	1	0100	0000000000010000	$A \cdot B$
1	1	0	1	0101	0000000000100000	$A \oplus B$
1	1	0	1	0110	0000000001000000	$A + B$
1	1	0	1	0111	0000000010000000	Not($A \oplus B$)

Table of Microcode's generated by Problem 3 (ALU1, FSM2)



Problem 3 Waveform (ALU1, FSM2)

y = even n = odd

State	Student Number	Id_sign	y[15..0]
0000	0101	n	0000000000000001
0001	0000	y	0000000000000010
0010	0001	n	0000000000000100
0011	0011	n	0000000000001000
0100	0010	y	0000000000010000
0101	0010	y	0000000000100000
0110	0111	n	0000000001000000
0111	0100	y	0000000010000000
1000	1000	y	0000000100000000

Problem 3 expected output (ALU1, FSM2)

Conclusion

In conclusion, this report effectively demonstrated how you can use multiple digital components together. The latches, FSM, ALU, and decoder all work together to implement a simple CPU. By designing, simulating and analysing problem sets 1 and 2, we were able to learn the powers of the ALU and how changing its conditions changes its output and how you can build off of simple ALU codes. In problem 3, we learned how we can use the FSM to alter the states and information which are microcoded by the decoder and fed into the ALU. The waveforms for the problem sets confirmed that all the components were interacting properly and had the correct code. We achieved the expected outputs and microcode values. Overall, our outputs effectively met the requirements of this lab.